



Using Python DB-API

Version 2026.1
2026-04-20

Using Python DB-API

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Introduction to Python DB-API for InterSystems IRIS	1
1.1 Usage	1
2 Python DB-API Quick Reference	3
2.1 Globals	3
2.2 Connection Class	4
2.2.1 Creating a Connection Object	4
2.2.2 Connection Class Methods	5
2.3 Cursor Class	6
2.3.1 Creating a Cursor object	6
2.3.2 Cursor attributes	7
2.3.3 Cursor methods	7
2.3.4 Cursor.description <i>type_code</i> Types and Values	11

1

Introduction to Python DB-API for InterSystems IRIS

See the [Table of Contents](#) for a detailed listing of the subjects covered in this document.

The InterSystems Python DB-API driver is a fully compliant implementation of the [PEP 249 version 2.0](#) Python Database API specification.

Note: **DB-API Installation**

The InterSystems DB-API driver is implemented as a subclass of the Python Native SDK which must be installed to use DB-API. See [Native Python SDK Installation and Setup](#) for details. Once you have installed the Native SDK, no special setup is required to use DB-API. See the code in the following section for an example of how to declare and connect to the DB-API driver.

1.1 Usage

The following example makes a connection to the InterSystems IRIS database, creates a cursor associated with the connection, sets up to make some DB-API calls, and then shuts down.

See the [Python DB-API Quick Reference](#) for detailed documentation on all implemented DB-API methods and attributes.

Connecting to the DB-API driver and getting a cursor

Python

```
import iris
import iris.dbapi

def main():
    connection_string = "localhost:1972/USER"
    username = "_system"
    password = "SYS"

    ssl_config = "MyConfig"

    connection = iris.dbapi.connect(connection_string, username, password, sslconfig=ssl_config)
    cursor = connection.cursor()

    try:
        pass # do something with DB-API calls
    except Exception as ex:
        print(ex)
    finally:
```

```
        if cursor:
            cursor.close()
        if connection:
            connection.close()

if __name__ == "__main__":
    main()
```

See [iris.dbapi.connect\(\)](#), [Connection.close\(\)](#), [Connection.cursor\(\)](#), and [Cursor.close\(\)](#) for more information on the methods called in this example. See [TLS with Python Clients](#) for information on setting up secure connections.

2

Python DB-API Quick Reference

The InterSystems Python DB-API driver is a fully compliant implementation of the [PEP 249 version 2.0](#) Python Database API specification. The following sections list all required implementation features, indicate the level of support for each one, and describe all InterSystems-specific features in detail:

- [Globals](#) lists values for required global constants *apilevel*, *threadsafety*, and *paramstyle*.
- [Connection Class](#) describes Connection methods [connect\(\)](#), [close\(\)](#), [commit\(\)](#), [rollback\(\)](#), and [cursor\(\)](#).
- [Cursor Class](#) describes the following Cursor members:
 - Attributes [arraysize](#), [description](#), and [rowcount](#).
Standard methods [callproc\(\)](#), [close\(\)](#), [execute\(\)](#), [executemany\(\)](#), [fetchone\(\)](#), [fetchmany\(\)](#), [fetchall\(\)](#), [nextset\(\)](#), [scroll\(\)](#), [setinputsizes\(\)](#), and [setoutputsize\(\)](#).
 - InterSystems extension methods [isClosed\(\)](#), [getTimeout\(\)](#), and [setTimeout\(\)](#).
 - Valid types and values for [Cursor.description](#)

2.1 Globals

These are required implementation-specific constants. In the InterSystems implementation, these globals are set to the following values:

apilevel

"2.0" — specifies compliance with PEP 249 version 2.0.

threadsafety

1 — may use DB-API in a multi-threaded application as long as threads do not share connections or cursors.

paramstyle

This is a global in the `_DBAPI.py` file that indicates the format of the input parameters when parameterized calls to `execute()`, `executemany()`, and `callproc()` are invoked with SQL statements. The following values are supported:

- "qmark" — query parameters use question mark style (for example: `WHERE name=?`).
- "named" — query parameters use named style (for example: `WHERE name=:name`).

The default value is `qmark`. The following example sets it to `named`:

```
import iris
iris.dbapi._DBAPI.paramstyle = "named"
```

If you are using InterSystems DB-API version 5.3 or later, this parameter does not have to be set. Either parameter style will be recognized regardless of the *paramstyle* setting. The following examples demonstrate correct syntax for each style:

Examples using "qmark"

An input parameter is indicated by a question mark (?). The input parameters are provided as a Python list. In InterSystems DB-API version 5.3 or later, "qmark" parameters will work even if *paramstyle* is not set.

```
sql = "Select * from Sample.Person where id = ? and name = ?"
params = [1, 'Jane Doe']
cursor.execute(sql, params) // same for direct_execute

sql = "Insert into Sample.Person (name, phone) values (?, ?)"
params = [('ABC', '123-456-7890'), ('DEF', '234-567-8901'), ('GHI', '345-678-9012')]
cursor.executemany(sql, params) // batch update

proc = "{ ? = Proc1(?) }" // parameter modes: RETURN_VALUE, INPUT
params = [1]
return_args = cursor.callproc(proc, params) // stored procedure

proc = "{ CALL Proc2 (?, ?) }" // parameter modes: INPUT, OUTPUT
params = ['abc']
return_args = cursor.callproc(proc, params) // stored procedure
```

Examples using "named"

An input parameter is indicated by a variable name preceded by a colon. The input parameters are provided as a Python dictionary where the keys indicate the variable names used in the SQL and the values for the keys hold the actual data for the corresponding variables/parameters. Named parameters will work even if *paramstyle* is not set.

```
sql = "SELECT * FROM Sample.Person WHERE firstname = :fname AND lastname = :lname"
params = {'fname' : 'John', 'lname' : 'Doe'}
cursor.execute(sql, params)

sql = "INSERT INTO Sample.Person(name) VALUES (:name)"
params = [{'name' : 'John'}, {'name' : 'Jane'}]
cursor.executemany(sql, params)
```

2.2 Connection Class

This section describes how to use `iris.dbapi.connect()` to create a Connection object, and provides implementation details for required Connection methods `close()`, `commit()`, `rollback()`, and `cursor()`.

See the [Usage](#) section in the Introduction for more information on connecting with DB-API.

2.2.1 Creating a Connection Object

DB-API Connection objects are created by calls to the `iris.dbapi.connect()` method. This method is identical to the Python Native SDK package method `iris.connect()` except that it can raise a DB-API exception.

Note: The `iris.dbapi` module supports required exception classes `Warning`, `Error`, `InterfaceError`, `DatabaseError`, `DataError`, `OperationalError`, `IntegrityError`, `InternalError`, `ProgrammingError`, and `NotSupportedError`.

iris.dbapi.connect()

`iris.dbapi.connect()` returns a new `Connection` object and attempts to create a new connection to an instance of InterSystems IRIS. The object will be open if the connection was successful, or closed otherwise (see `Cursor.isClosed()`).

```
iris.dbapi.connect(hostname,port,namespace,username,password,timeout,sharedmemory,logfile)
iris.dbapi.connect(connectionstr,username,password,timeout,sharedmemory,logfile)
```

The *hostname*, *port*, *namespace*, *timeout*, and *logfile* from the last successful connection attempt are saved as properties of the connection object.

parameters:

Parameters may be passed by position or keyword.

- `hostname` — str specifying the server URL. If this is not `localhost`, then the `sharedmemory` parameter is set to `False`.
- `port` — int specifying the superserver port number
- `namespace` — str specifying the namespace on the server
- The following parameter can be used in place of the `hostname`, `port`, and `namespace` arguments:
 - `connectionstr` — str of the form `hostname:port/namespace`.
- `username` — str specifying the user name
- `password` — str specifying the password
- `timeout` (optional) — int specifying maximum number of seconds to wait while attempting the connection. Defaults to 10.
- `sharedmemory` (optional) — specify bool `True` to attempt a shared memory connection when the hostname is `localhost` or `127.0.0.1`. Specify `False` to force a connection over TCP/IP. In many cases, this defaults to `True`; however, if the hostname is not local or if you have specified an SSL configuration, it is set to `False` and cannot be overridden.
- `logfile` (optional) — str specifying the client-side log file path. The maximum path length is 255 ASCII characters. Defaults to `None`
- `sslconfig` — The name of the SSL configuration defined in the `SSLDefs.ini` file that should be used to secure the connection. Defaults to `None`. If you specify a value for this parameter, then the `sharedmemory` parameter is set to `False`.

2.2.2 Connection Class Methods

A `Connection` object can be used to create one or more `Cursor` objects. Database changes made by one cursor are immediately visible to all other cursors created from the same connection. Rollbacks and commits affect all changes made by cursors using this connection.

close()

`Connection.close()` closes the connection immediately. The connection and all cursors associated with it will be unusable. An implicit rollback will be performed on all uncommitted changes made by associated cursors.

```
Connection.close()
```

A `ProgrammingError` exception will be raised if any operation is attempted with a closed connection or any associated cursor.

commit()

`Connection.commit()` commits all SQL statements executed on the connection since the last commit/rollback. The rollback affects all changes made by any cursor using this connection. Explicit calls to this method are not required.

```
Connection.commit()
```

rollback()

`Connection.rollback()` rolls back all SQL statements executed on the connection that created this cursor (since the last commit/rollback). It affects all changes made by any cursor using this connection.

```
Connection.rollback()
```

cursor()

`Connection.cursor()` returns a new `Cursor` object that uses this connection.

```
Connection.cursor()
```

Any changes made to the database by one cursor are immediately visible to all other cursors created from the same connection. Rollbacks and commits affect all changes made by any cursor using this connection.

2.3 Cursor Class

This section describes how to [create a Cursor object](#), and provides implementation details for the following required `Cursor` methods and attributes:

- Attributes [arraysize](#), [description](#), and [rowcount](#).
- Standard methods [callproc\(\)](#), [close\(\)](#), [execute\(\)](#), [executemany\(\)](#), [fetchone\(\)](#), [fetchmany\(\)](#), [fetchall\(\)](#), [nextset\(\)](#), [scroll\(\)](#), [setinputsizes\(\)](#), and [setoutputsize\(\)](#).
- InterSystems extension methods [isClosed\(\)](#), [getTimeout\(\)](#), and [setTimeout\(\)](#).
- Valid types and values for [Cursor.description](#)

2.3.1 Creating a Cursor object

A `Cursor` object is created by establishing a connection and then calling `Connection.cursor()`. For example:

```
connection = iris.connect(connection_string, username, password)
cursor = connection.cursor()
```

Any changes made to the database by one cursor are immediately visible to all other cursors created from the same connection.

Once the cursor is closed, accessing the column data of a row (`DataRow` object) or any other functions of the `Cursor` class will result in an error.

See the [Usage](#) section in the Introduction for a more complete example. See [Creating a Connection Object](#) for detailed information on creating a connection.

2.3.2 Cursor attributes

arraysize

Cursor.**arraysize** is a read/write attribute that specifies the number of rows to fetch at a time with [fetchmany\(\)](#). Default is 1 (fetch one row at a time). Return value on a closed object is `arraysize = 1`.

description

Cursor.**description** returns a list of tuples containing information for each result column returned by the last SQL select statement. Value will be *None* if an execute method has not been called, if the last operation did not return any rows, or if the Cursor object is closed.

Each tuple (column description) in the list contains the following items:

- *name* — column name (required, defaults to *None*)
- *type_code* — integer SQLType identifier (required, defaults to 0). See [Cursor.description Types and Values](#) for valid values.
- *display_size* — not used - value set to *None*
- *internal_size* — not used - value set to *None*
- *precision* — integer (optional, defaults to 0)
- *scale* — integer (optional, defaults to *None*)
- *nullable* — integer (optional, defaults to 0)

rowcount

Cursor.**rowcount** specifies the number of rows modified by the last SQL statement. The value will be -1 if no SQL has been executed, if the number of rows is unknown, or if the Cursor is closed.

For example, DDLs like CREATE, DROP, DELETE, and SELECT statements (for performance reasons) return -1.

Batch updates also return the number of rows affected.

2.3.3 Cursor methods

callproc()

Cursor.**callproc()** calls a stored database procedure with the given *procname*.

```
Cursor.callproc(procname)
Cursor.callproc(procname, parameters)
```

parameters:

- *procname* – string containing a stored procedure call with parameterized arguments.
- *parameters* – list of parameter values to pass to the stored procedure

Any of the `fetch*()` methods can be used to access the rows of a result set for a stored procedure that is expected to return result sets. They are expected to behave in the same way as for SELECT queries (non-procedures). For example, after using **callproc()** to call a procedure that is expected to return at least one result set, **fetchone()** will return the first row of the first result set and subsequent calls to **fetchone()** will return the remaining rows one by one. A **fetchall()** call will return all the remaining rows of the current result set.

For example, this code calls stored procedure `Sample.SP_Sample_By_Name`, specifying parameter value "A" in a list:

```
cursor.callproc("CALL Sample.SP_Sample_By_Name (?)", ["A"])
row = cursor.fetchone()
while row:
    print(row.ID, row.Name, row.DOB, row.SSN)
    row = cursor.fetchone()
```

Output will be similar to the following:

```
167 Adams,Patricia J. 1964-10-12 216-28-1384
28 Ahmed,Dave H. 1954-01-12 711-67-4091
20 Alton,Samantha E. 2015-03-28 877-53-4204
118 Anderson,Elvis V. 1994-05-29 916-13-245
```

If a stored procedure of 'function' type is expected to return a result set, then it will be available in the return value of **callproc()** as a tuple at the corresponding placeholder. An internal call to **fetchall()** is made in this specific case, hence, the tuple holds all the rows of the result set. The rows hold all the column data as well.

For example, the stored procedure below has two parameters whose modes are RETURN_VALUE and INPUT, respectively:

```
proc = "{ ? = MyProc3(?) }"
params = [1]
return_args = cursor.callproc(proc, params)
print(return_args[0]) # tuple of all the result set rows
print(return_args[1] == None)
```

Note: Outputs of **fetch*()** and **callproc()** APIs which returned Python lists in previous releases now return Python tuples. If the output from older version was `[1, "hello"]`, then the new version will return `(1, "hello")`. Python tuples containing one item will have a comma appended. For example, a list such as `[100]` is represented as tuple `(100, )`.

close()

Cursor.**close()** closes the cursor.

```
Cursor.close()
```

A `ProgrammingError` exception will be raised if any operation is attempted with a closed cursor. Cursors are closed automatically when they are deleted (typically when they go out of scope), so calling this is not usually necessary.

execute()

Cursor.**execute()** executes the query specified in the *operation* parameter. Updates the Cursor object and sets the `rowcount` attribute to -1 for a query or 1 for an update.

```
Cursor.execute(operation)
Cursor.execute(operation, parameters)
```

parameters:

- `operation` – string containing SQL statement to be executed
- `parameters` – optional list or tuple of values. This must be a Python list or tuple (sets are not acceptable).

examples:

Parameter values are used in positions where the SQL statement contains a ? (qmark) rather than a literal or constant. If the statement does not contain any qmarks, the *parameters* argument is not required will raise an exception if given.

- `sql = "...(1,2)..."; execute(sql)`
- `sql = "...(?,...)"; params = [1,2]; execute(sql, params)`
- `sql = "...(1,...)"; params = [2]; execute(sql, params)`

executemany()

`Cursor.executemany()` is used for batch inserts/updates/deletes. It prepares a database operation (query or command) and then executes it against all parameter sequences or mappings found in the sequence *seq_of_parameters*.

```
Cursor.executemany(operation)
Cursor.executemany(operation, seq_of_parameters)
```

parameters:

- `operation` – string containing SQL INSERT or UPDATE statement to be executed
- `seq_of_parameters` – sequence of parameter sequences or mappings

Returns a tuple of integers and/or strings, depending on success or failure of the update operation. Each item in the tuple corresponds to the row in the batch which is a list/tuple of user-provided parameters. (Integers for success, strings for error messages in case of failure). Returns 1 to indicate every successful INSERT or an error message with details in case of a failure. Returns a cardinal number to indicate the number of rows that were successfully modified for an UPDATE or an error message with details in case of failure. The [rowcount](#) attribute indicates the number of rows successfully inserted/updated/deleted.

fetchone()

`Cursor.fetchone()` returns the pointer to the next `iris.dbapi..DataRow` object (internal/InterSystems-specific) in the query, or `None` if no more data is available.

```
Cursor.fetchone()
```

Data is fetched only on request, via indexing. Index values must be positive integers (a value of 0 refers to column 1, a value of 1 refers to column 2, and so on).

Column values can be fetched using cardinal values, column name (as a string), and the slice operator, but not via dynamic attributes. For example, `row[:]` fetches all the column data, `row[0]` fetches the data in first column, `row[1]` fetches the data in second column, and `row['<columnName>']` fetches the data from the `<columnName>` column, but `row.columnName` will not work.

A `ProgrammingError` exception is raised if no SQL has been executed or if it did not return a result set (for example, if it was not a SELECT statement).

fetchmany()

`Cursor.fetchmany()` fetches the next set of rows of a query result, returning a sequence of sequences (a tuple of tuples). If the *size* argument is not specified, the number of rows to fetch at a time is set by the `Cursor.arraysize` attribute (default 1). An empty sequence is returned when no more rows are available.

```
Cursor.fetchmany()
Cursor.fetchmany(size)
```

parameters:

- `size` – optional. Defaults to the current value of attribute `Cursor.arraysize`.

fetchall()

Cursor.**fetchall()** fetches all remaining rows of a query result.

```
Cursor.fetchall()
```

getTimeout() [InterSystems extension method]

Cursor.**getTimeout()** returns an integer value corresponding to the current query timeout (the number of seconds to wait for a SQL statement to be executed before returning an error to the application). A value of 0 indicates no timeout (query will wait indefinitely for completion).

```
Cursor.getQueryTimeout()
```

Also see [setTimeout\(\)](#)

isClosed() [InterSystems extension method]

Cursor.**isClosed()** is an InterSystems extension method that returns `True` if the cursor object is already closed, `False` otherwise.

```
Cursor.isClosed()
```

nextset()

Cursor.**nextset()** is a DB-API method for iterating over multiple result sets. It makes the cursor skip to the next available set, discarding any remaining rows from the current set. If there are no more sets, the method returns `None`. Otherwise, it returns `True` and subsequent calls to the **fetch*()** methods will return rows from the next result set.

```
Cursor.nextset()
```

Example:

```
proc = "CALL Sample.Procedure(?)"
params = [<val>]
cursor.callproc(proc, params)
rows = cursor.fetchall() # returns all rows in the first result set
newRsSet = cursor.nextset()
while (newRsSet):
    rows = cursor.fetchall() # returns all rows in the subsequent result set (if available)
    newRsSet = cursor.nextset()
```

scroll()

Cursor.**scroll()** is a DB-API method that scrolls the cursor in the result set to a new position and returns the row at that position. This method does not work with stored procedures. It raises an `IndexError` if scroll operation would leave the result set.

```
Cursor.scroll(value, mode)
```

parameters:

- **value** – integer value specifying the new target position.
 - If *mode* is `relative` (the default), *value* is a positive or negative offset to the current position in the result set.
 - If *mode* is `absolute`, *value* is an absolute target position (negative values are not valid).
- **mode** – optional. Valid values are `relative` or `absolute`. The use of an empty string for the *mode* argument sets its value to `relative` (for example, `cursor.scroll(3, '')`).

Example:

For each example, assume the result set has a total of 10 rows, and the initial number of rows fetched is 5. Result set index values are 0-based, so the current position in the result set is `rs[4]` (the 5th row).

```
cursor.execute("select id, * from simple.human where id <= 10")
cursor.fetchmany(5)

# Scroll forward 3 rows, relative to Row 5
datarow = cursor.scroll(3, 'relative')      # Row 8
print(datarow[0] == 8)

# Scroll to absolute position 3
datarow = cursor.scroll(3, 'absolute')      # Row 3
print(datarow[0] == 3)

# Scroll backward 4 rows, relative to Row 3 (mode defaults to 'relative')
datarow = cursor.scroll(-4, '')             # Row 9
print(datarow[0] == 9)

# Attempt to scroll to absolute position -4
# Error: Negative values with absolute scrolling are not allowed.
datarow = cursor.scroll(-4, 'absolute')     # ERROR
print(datarow[0])
```

setinputsizes()

`Cursor.setinputsizes()` is not applicable to InterSystems IRIS, which does not implement or require this functionality. Throws `NotImplementedError` if called.

setoutputsize()

`Cursor.setoutputsize()` is not applicable to InterSystems IRIS, which does not implement or require this functionality. Throws `NotImplementedError` if called.

setTimeout() [InterSystems extension method]

`Cursor.setTimeout()` sets the number of seconds the driver will wait for the SQL statement to be executed before terminating the operation and giving an error. By default, there is no limit on the amount of time allowed for a running SQL statement to complete. If the limit is exceeded, an `OperationalError` is thrown.

```
Cursor.setQueryTimeout(int)
```

parameters:

- `int` – integer number of seconds to wait before the operation times out.

Also see [getTimeout\(\)](#)

2.3.4 `Cursor.description type_code` Types and Values

A `Cursor` object's **description** attribute returns a list of tuples containing information about each of the result columns of a query. This section lists the valid [SQLType Values](#) and [Type Objects](#) associated with the `type_code` tuple.

2.3.4.1 Valid `type_code` SQLType Values

Valid `SQLType` enumeration values for the `Cursor.description type_code`.

- `BIGINT` = -5
- `BINARY` = -2
- `BIT` = -7
- `CHAR` = 1

- `DECIMAL` = 3
- `DOUBLE` = 8
- `FLOAT` = 6
- `GUID` = -11
- `INTEGER` = 4
- `LONGVARBINARY` = -4
- `LONGVARCHAR` = -1
- `NUMERIC` = 2
- `REAL` = 7
- `ROWID` = -12
- `SMALLINT` = 5
- `DATE` = 9
- `TIME` = 10
- `TIMESTAMP` = 11
- `TINYINT` = -6
- `TYPE_DATE` = 91
- `TYPE_TIME` = 92
- `TYPE_TIMESTAMP` = 93
- `VARBINARY` = -3
- `VARCHAR` = 12
- `WCHAR` = -8
- `WLONGVARCHAR` = -10
- `WVARCHAR` = -9
- `DATE_HOROLOG` = 1091
- `TIME_HOROLOG` = 1092
- `TIMESTAMP_POSIX` = 1093

2.3.4.2 Valid *type_code* Type Objects and Constructors

The *type_code* must compare equal to one of the Type Objects defined below. Type Objects may be equal to more than one type code (e.g. `DATETIME` could be equal to the type codes for date, time and timestamp columns).

The `iris.dbapi` module exports the following constructors and singletons:

Constructors

The following constructors are available for date, time, and binary (long) string values:

Date(year, month, day)

Constructs an object holding a date value

Time(hour, minute, second)

Constructs an object holding a time value

Timestamp(year, month, day, hour, minute, second)

Constructs an object holding a time stamp value

DateFromTicks(ticks)

Constructs an object holding a date value from the given ticks value (number of seconds since the epoch; see the documentation of the standard Python time module for details).

TimeFromTicks(ticks)

Constructs an object holding a time value from the given ticks value (number of seconds since the epoch; see the documentation of the standard Python time module for details).

TimestampFromTicks(ticks)

Constructs an object holding a time stamp value from the given ticks value (number of seconds since the epoch; see the documentation of the standard Python time module for details).

Binary(string)

Constructs an object capable of holding a binary (long) string value.

Singletons

The following singleton type objects are available:

STRING type

Used to describe columns in a database that are string-based (e.g. CHAR).

BINARY type

Used to describe (long) binary columns in a database (e.g. LONG, RAW, BLOBs).

NUMBER type

Used to describe numeric columns in a database.

DATETIME type

Used to describe date/time columns in a database.

ROWID type

Used to describe the “Row ID” column in a database.

Example using type objects

```
import iris
from iris.dbapi import (SQLType, STRING, BINARY, NUMBER, DATETIME, ROWID)

cursor.execute("SELECT * FROM Sample.Person")
column_info = cursor.description
print(column_info[0][0]) # displays the name of the 1st column
# Assuming the 1st column is of VARCHAR type (i.e., STRING type), the following lines will print
True
print(column_info[0][1] == SQLType.VARCHAR)
print(column_info[0][1] == STRING)
print(column_info[0][1] != NUMBER)
```

